

Rotina Inteligente Gerenciador de Atividades — Orientação a Objetos em Python

Augusto | Pedro · Princípios de Programação · 2026





CONTEXTO

Camila Ferreira — Perfil e Problema

Perfil

20 anos · 3º semestre de Psicologia

Estágio clínico (manhã) + aulas (tarde) + pesquisa (noite)

Academia 3×/semana

Problemas Identificados

- Perda de prazos por falta de visibilidade
- Ausência de priorização estruturada
- Sem visão semanal consolidada das atividades

Solução Proposta

Sistema OO em Python sob medida para gerenciar a rotina da Camila — modular, extensível e orientado a objetos.

SOLUÇÃO

O que o Sistema Entrega



Atividades como Objetos

Atributos: nome, categoria, duração, prioridade (1–5) e prazo



Detecção de Sobrecarga

Alerta automático quando carga diária ultrapassa 8 horas



Atividades Atrasadas

Comparação com `date.today()` para identificar prazos vencidos



Relatório Semanal

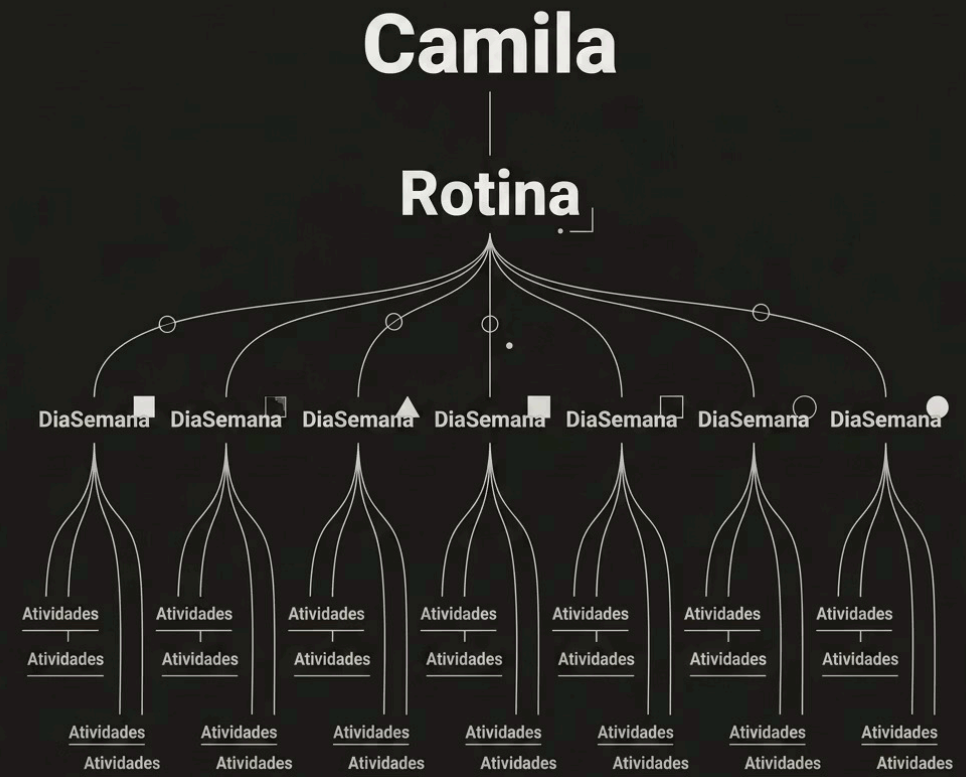
Urgentes, atrasadas e carga total consolidados por semana

Classes, Objetos e Arquitetura

Hierarquia do Projeto

Fundamentos

- **Classe** = molde que define atributos e comportamentos
- **Objeto** = instância concreta com valores próprios
- `self.nome` → variável de instância (por objeto)
- `total_atividades` → variável de classe (compartilhada)
- `__init__` = construtor, executado na instanciação



class Atividade — Construção do Objeto

Estrutura da Classe

- Construtor recebe: nome, categoria, duração, prioridade, prazo
- `concluir()` → altera estado: `self.concluida = True`
- `esta_atrasada()` → `date.today() > self.prazo`
- Contador de classe incrementado a cada instância criada

```
class Atividade:
    total_atividades = 0 # variável de CLASSE (compartilhada por todos)

    def __init__(self, nome, categoria, duracao_horas, prioridade, prazo=None):
        self.nome = nome # variável de INSTÂNCIA
        self.categoria = categoria
        self.duracao_horas = duracao_horas
        self.prioridade = prioridade # 1 (baixa) a 5 (urgente)
        self.prazo = prazo
        self.concluida = False
        Atividade.total_atividades += 1

    def concluir(self):
        self.concluida = True

    def esta_atrasada(self):
        if self.prazo and not self.concluida:
            return date.today() > self.prazo
        return False
```

❗ Cada atividade é um objeto independente com estado e comportamento próprios.

class DiaSemana — Composição de Objetos

Responsabilidades

- Contém lista de objetos `Atividade` → **composição**
- `carga_total()` → soma `duracao_horas` de pendentes
- `esta_sobrecarregado()` → `carga_total() > 8`
- `exibir()` → ordena por prioridade: `sorted(..., key=lambda x: -x.prioridade)`

☐ DiaSemana não herda de Atividade — ele **contém** atividades.
Composição sobre herança.

```
python)
11 ybhon cel) {
2 puboctlol(dxtr); {
11 pututsitovo="(Y) }
13 phacdemen, ct=3; {
16 winu=schr(plantlaactin, (')),
14 foycamldell; { (
16 }
13 olakbar kimmjub))
12 atarlion(Janpr 0. "") }
13 }
12 nanpwoncs kmholDef"irt((luo'o""+1) }
13 oneznecs(); {
36 }
19 puneriolnlcec("0."), )
74 obbycts ("
14 oyes "dd"="d")" }
11 }
11 vabeqlayous("=""(Y); )
73 octawlocs Chs":(0), 4f"); )
13 furswunesron.hondle();")
5 sosoacattxb-f; ""
4 prcowor canilbr("(kf"}f"); )
12 }
18 whprtouncat();
2 )
2 )
```

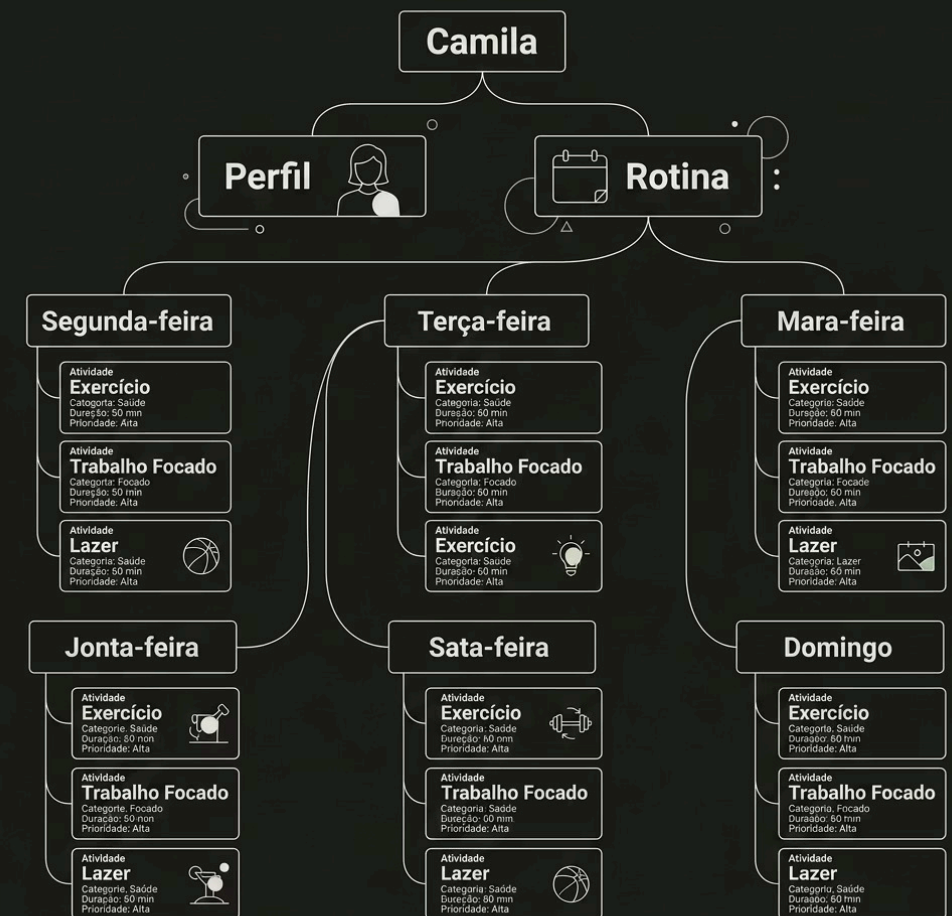
class Rotina e class Camila — Arquitetura Completa

class Rotina

- Instancia 7 objetos `DiaSemana` via *dict comprehension*
- `atividades_urgentes()` → varre todos os dias, filtra `prioridade ≥ 4`

class Camila

- Encapsula perfil + instância de Rotina
- `adicionar_atividade()` → cria objeto e delega ao dia correto



PEDRO · DEMO

Execução — python rotina_etapa1.py

01

Cabeçalho

Perfil da Camila + carga semanal total

02

Segunda-feira

Alerta **SOBRECARGADA** — 11h > 8h

03

Ordenação por Prioridade

Atividades do dia exibidas em ordem decrescente

04

⚠ Atividade Atrasada

Trabalho de Psicologia Social com prazo vencido

05

Relatório Final

Urgentes + atrasadas + totais consolidados

Resultados e Pilares OO Aplicados

Pilares Demonstrados

- **Classe / Objeto** — Atividade, DiaSemana, Rotina, Camila
- **Encapsulamento** — atributos e métodos colocalizados
- **Composição** — objetos aninhados formam hierarquia funcional
- **Variáveis de classe vs. instância** — `total_atividades` vs. `self.nome`

Resultado

Sistema funcional que resolve o problema real da Camila — código modular, extensível e legível.

Uso de IA

Estrutura e arquitetura auxiliadas por IA generativa — todo código revisado e compreendido pela equipe.